opentext™

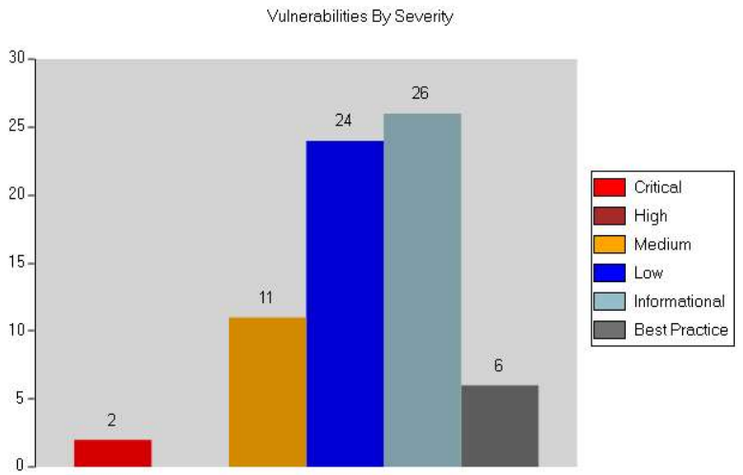
Fortify WebInspect

Vulnerability (Legacy)

Web Application Assessment Report

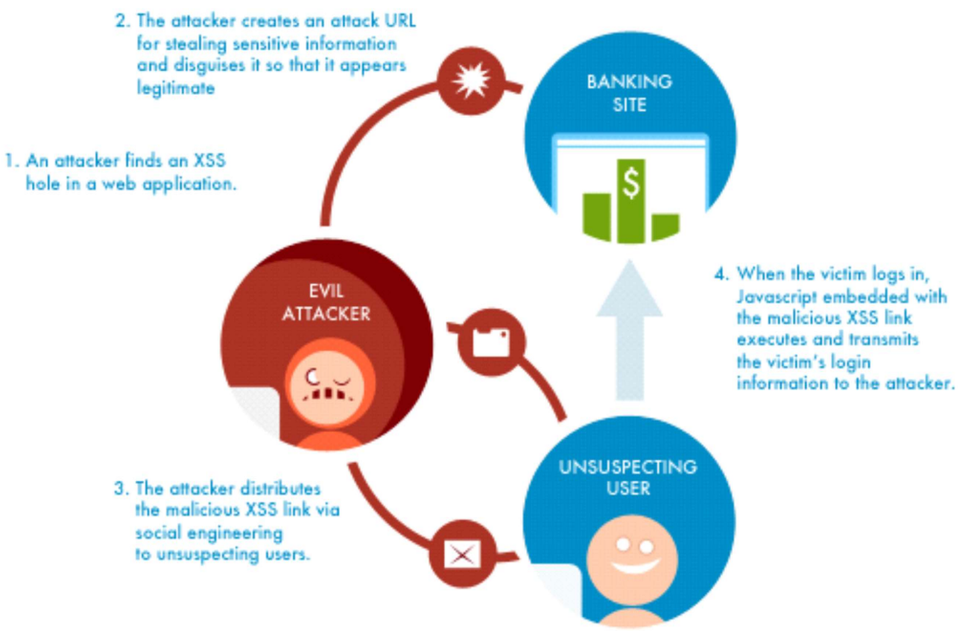
Scan Name:	Workflow:https://portalproyectos.reniec.gob.pe/PLPI		
Policy:	Standard	**Crawl Sessions:**	213
Scan Date:	4/6/2026 6:12:23 PM	**Vulnerabilities:**	37
Scan Version:	24.4.0.70	**Scan Duration:**	11 minutes : 41 seconds
Scan Type:	Site	**Client:**	Custom

Server: https://portalproyectos.reniec.gob.pe:443



Critical **Cross-Site Scripting: Reflected**

Summary:



Cross-Site Scripting vulnerability found in Post parameter idioma. The following attack uses plain encoding:

"><sCrIpT>alert(14727)</sCrIpT>

A Cross-Site Scripting (XSS) vulnerability was detected in the web application. Cross-Site Scripting occurs when dynamically generated web pages display user input, such as login information, that is not properly validated, allowing an attacker to embed malicious scripts into the generated page and then execute the script on the machine of any user that views the site. In this instance, the web application was vulnerable to an automatic payload, meaning the user simply has to visit a page to make the malicious scripts execute. If successful, Cross-Site Scripting vulnerabilities can be exploited to manipulate or steal cookies, create requests that can be mistaken for those of a valid user, compromise confidential information, or execute malicious code on end user systems. Recommendations include implementing secure programming techniques that ensure proper filtration of user-supplied data, and encoding all user supplied data to prevent inserted scripts being sent to end users in a format that can be executed.

Execution:

View the attack string included with the request to check what to search for in the response. For instance, if "(javascript:alert ('XSS')" is submitted as an attack (or another scripting language), it will also appear as part of the response. This indicates that the web application is taking values from the HTTP request parameters and using them in the HTTP response without first removing potentially malicious data.The response can be viewed in "Web Browser" view in the Vulnerability pane to see the injected popup events in action. Events requiring user interaction (e.g. onmouseover or onclick events) can be triggered by performing the corresponding action (e.g. clicking the injected link). Injection with numeric string in src, or href, attributes indicates that the site is vulnerable to script include or content exfiltration. These can be verified by repeating the request in a browser and intercepting originating network traffic in a web proxy.

Implication:

Cross-Site Scripting(XSS) happens when user input from a web client is immediately included via server-side scripts in a dynamically generated web page. Reflected XSS is specifically considered critical when malicious payload can be embedded in a URL (e.g. in query strings of GET requests). An attacker can trick a victim, via phishing attack, to click on a link with vulnerable input which has been altered to include attack code and then sent to the legitimate server. The injected code is then reflected back to the user's browser which executes it.

The implications of successful Cross-Site Scripting attacks are:

- Account hijacking - An attacker can hijack the user's session before the session cookie expires and take actions with the privileges of the user who accessed the URL, such as issuing database queries and viewing the results.
- Malicious script execution - Users can unknowingly execute JavaScript, VBScript, ActiveX, HTML, or even Flash content that has been inserted into a dynamically generated page by an attacker.
- Worm propagation - With Ajax applications, XSS can propagate somewhat like a virus. The XSS payload can autonomously inject itself into pages, and easily re-inject the same host with more XSS, all of which can be done with no hard refresh. Thus, XSS can send multiple requests using complex HTTP methods to propagate itself invisibly to the user.
- Information theft - Via redirection and fake sites, attackers can connect users to a malicious server of the attacker's choice and capture any information entered by the user.
- Denial of Service - Often by utilizing malformed display requests on sites that contain a Cross-Site Scripting vulnerability, attackers can cause a denial of service condition to occur by causing the host site to query itself repeatedly .
- Browser Redirection - On certain types of sites that use frames, a user can be made to think that he is in fact on the original site when he has been redirected to a malicious one, since the URL in the browser's address bar will remains the same. This is because the entire page isn't being redirected, just the frame in which the JavaScript is being executed is redirected.
- Manipulation of user settings - Attackers can change user settings for nefarious purposes.
- Bypass Content-Security-Policy protection - Attackers can inject a malformed tag formation, known as dangling tag injection, which in some cases allows injected script to reuse valid nonce on the page and bypass script source restriction. Additionally dangling tag injection can be used to steal sensitive information embedded in HTML response if browser is able to make a request to the injected link.
- Base tag injection: Attacker can cause relative links on a page to load from a different domain by modifying the base URL for the page via base tag injection.
- Link prefetch injection: While unable to execute script, attackers can use link tag with rel=prefetch that will make browsers pre-fetch the specified link even though it is never rendered and rejected subsequently due to web application enforced cross-site policy (e.g. CSP protections).
- Edge side includes (ESI) Injection - ESI is a markup language used in various HTTP devices, such as reverse proxies and load balancers, that are positioned between client and server. An attacker can inject ESI markup to perform critical attacks such as cross-site scripting and HTTPOnly cookie protection bypass.

Fix:

For Development:

Cross-Site Scripting attacks can be avoided by carefully validating all input, and properly encoding all output. When validating user input, verify that it matches the strictest definition of valid input possible. For example, if a certain parameter is supposed to be a number, attempt to convert it to a numeric data type in your programming language.

PHP: intval("0".$_GET['q']);

ASP.NET: int.TryParse(Request.QueryString["q"], out val);

The same applies to date and time values, or anything that can be converted to a stricter type before being used. When accepting other types of text input, make sure the value matches either a list of acceptable values (white-listing), or a strict regular expression. If at any point the value appears invalid, do not accept it. Also, do not attempt to return the value to the user in an error message.

Most server side scripting languages provide built in methods to convert the value of the input variable into correct, non-interpretable HTML. These should be used to sanitize all input before it is displayed to the client.

PHP: string htmlspecialchars (string string [, int quote_style])

ASP.NET: Server.HTMLEncode (strHTML String)

When reflecting values into JavaScript or another format, make sure to use a type of encoding that is appropriate. Encoding data for HTML is not sufficient when it is reflected inside of a script or style sheet. For example, when reflecting data in a JavaScript string, make sure to encode all non-alphanumeric characters using hex (\xHH) encoding.

If you have JavaScript on your page that accesses unsafe information (like location.href) and writes it to the page (either with document.write, or by modifying a DOM element), make sure you encode data for HTML before writing it to the page. JavaScript does not have a built-in function to do this, but many frameworks do. If you are lacking an available function, something like the following will handle most cases:

s = s.replace(/&/g,'&').replace(/"/i,'"').replace(/</i,'<').replace(/>/i,'>').replace(/'/i,''')

Ensure that you are always using the right approach at the right time. Validating user input should be done as soon as it is received. Encoding data for display should be done immediately before displaying it.

For Security Operations:

Server-side encoding, where all dynamic content is first sent through an encoding function where Scripting tags will be replaced with codes in the selected character set, can help to prevent Cross-Site Scripting attacks.

Many web application platforms and frameworks have some built-in support for preventing Cross-Site Scripting. Make sure that any built-in protection is enabled for your platform. In some cases, a misconfiguration could allow Cross-Site Scripting. In ASP.NET, if a page's EnableViewStateMac property is set to False, the ASP.NET view state can be used as a vector for Cross-Site Scripting.

An IDS or IPS can also be used to detect or filter out XSS attacks. Below are a few regular expressions that will help detect Cross-Site Scripting.

Regex for a simple XSS attack:
/((\%3C) <)((\%2F) \/)*[a-z0-9\%]+((\%3E) >)/ix

The above regular expression would be added into a new Snort rule as follows:

alert tcp $EXTERNAL_NET any -> $HTTP_SERVERS $HTTP_PORTS (msg:"NII Cross-Site Scripting attempt"; flow:to_server,established; pcre:"/((\%3C) <)((\%2F) \/)*[a-z0-9\%]+((\%3E) >)/i"; classtype:Web-application-attack; sid:9000; rev:5;)

Paranoid regex for XSS attacks:
/((\%3C) <)[^\n]+((\%3E) >)/I

This signature simply looks for the opening HTML tag, and its hex equivalent, followed by one or more characters other than the new line, and then followed by the closing tag or its hex equivalent. This may end up giving a few false positives depending upon how your web application and web server are structured, but it is guaranteed to catch anything that even remotely resembles a Cross-Site Scripting attack.

For QA:

Fixes for Cross-Site Scripting defects will ultimately require code based fixes. Read the the following links for more information about manually testing your application for Cross-Site Scripting.

Reference:

OWASP Cross-Site Scripting Information
https://www.owasp.org/index.php/XSS

CERT
http://www.cert.org/advisories/CA-2000-02.html

Apache
http://httpd.apache.org/info/css-security/apache_specific.html

CGISecurity
https://www.cgisecurity.com/xss-faq.html

File Names:

- https://portalproyectos.reniec.gob.pe:443/PLPI/loginDatosDni
- https://portalproyectos.reniec.gob.pe:443/PLPI/loginDatosDni

---

Medium **Poor Error Handling: Unhandled Exception**

Summary:

A Java runtime error message was found, indicating that the assessment has generated an unhandled exception in your Web application code. Unhandled exceptions are circumstances in which the application has received user input (or parameters) that it did not expect and does not know how to handle. In many cases, an attacker can leverage the conditions that cause

these errors in order to gain unauthorized access to the system. Recommendations include providing logic to handle any situation in which a parameter is not being passed or is being passed incorrectly.

Implication:

The error message may contain the location of the file in which the offending function is located. This may disclose the webroot's absolute path, as well as give the attacker the location of application include files or configuration information. It may even disclose the portion of code that failed.

Note: This vulnerability may be a false positive if the page it is flagged on is technical documentation.

Fix:

For Security Operations:

Ultimately, this problem will require a change to the application code. However, follow these recommendations to help ensure a secure Web application:

- **Use Uniform Error Codes:** Ensure that you are not inadvertently supplying information to an attacker via the use of inconsistent or "conflicting" error messages. For instance, don't reveal unintended information by using error messages such as Access Denied, which will also let an attacker know that the file he seeks actually exists. Use consistent terminology for files and folders that do exist, do not exist, and which have read access denied.
- **Informational Error Messages:** Ensure that error messages do not reveal too much information. Complete or partial paths, variable and file names, row and column names in tables, and specific database errors should never be revealed to the end user. Remember, an attacker will gather as much information as possible, and then add pieces of seemingly innocuous information together to craft an attack.
- **Proper Error Handling:** Use generic error pages and error handling logic to inform end users of potential problems. Do not provide system information or other data that could be used by an attacker when orchestrating an attack.

For Development:

This problem arises from the improper validation of characters that are accepted by the application. Any time a parameter is passed into a dynamically-generated web page, you must assume that the data could be incorrectly formatted. The application should contain sufficient logic to handle any situation in which a parameter is not being passed or is being passed incorrectly. Keep in mind how the data is being submitted, as a result of a GET or a POST. Additionally, to develop secure and stable code, treat cookies the same as parameters. The following recommendations will help ensure that you are delivering secure Web applications.

- **Stringently define the data type:** Stringently define the data type (a string, an alphanumeric character, etc.) that the application will accept. Validate input for improper characters. Adopt the philosophy of using what is good rather than what is bad. Define the allowed set of characters. For instance, if a field is to receive a number, allow that field to accept only numbers. Define the maximum and minimum data lengths that the application will accept.
- **Verify parameter is being passed:** If a parameter that is expected to be passed to a dynamic Web page is omitted, the application should provide an acceptable error message to the user. Also, never use a parameter until you have verified that it has been passed into the application.
- **Verify correct format:** Never assume that a parameter is of a valid format. This is especially true if the parameter is being passed to a SQL database. Any string that is passed directly to a database without first being checked for proper format can be a major security risk. Also, just because a parameter is normally provided by a combo box or hidden field, do not assume the format is correct. A hacker will first try to alter these parameters while attempting to break into your site.
- **Verify file names being passed in via a parameter:** If a parameter is being used to determine which file to process, never use the file name before it is verified as valid. Specifically, test for the existence of characters that indicate directory traversal, such as .../, c:\, and /.
- **Do not store critical data in hidden parameters:** Many programmers make the mistake of storing critical data in a hidden parameter or cookie. They assume that since the user doesn't see it, it's a good place to store data such as price, order number, etc. Both hidden parameters and cookies can be manipulated and returned to the server, so never assume the client returned what you sent via a hidden parameter or cookie.

For QA:

Simple testing can usually determine how your web application will react to different input errors. More expansive testing must be conducted to cause internal errors to gauge the reaction of the site. This assessment performs both of these tasks.

Ensure that the error handling scheme is consistent and does not reveal private information about your Web application. A seemingly innocuous piece of information can provide an attacker the means to discover additional information that can be used to conduct an attack. Make the following observations:

- Do you receive the same type of error for existing and non-existing files?
- Does the error include phrases (such as "Permission Denied") that could reveal the existence of a file?

File Names:

- https://portalproyectos.reniec.gob.pe:443/PLPI/consultaPadron/cargar/
- https://portalproyectos.reniec.gob.pe:443/PLPI/consultaPadron/cargar/
- https://portalproyectos.reniec.gob.pe:443/PLPI/consultaPadron/cargar/
- https://portalproyectos.reniec.gob.pe:443/PLPI/consultaPadron/cargar/
- https://portalproyectos.reniec.gob.pe:443/PLPI/consultaPadron/cargar/
- https://portalproyectos.reniec.gob.pe:443/PLPI/consultaPadron/cargar/

Medium **Cookie Security: Cookie not Sent Over SSL**

Summary:

This policy states that any area of the website or web application that contains sensitive information or access to privileged functionality such as remote site administration requires that all cookies are sent via SSL during an SSL session. The URL: https://portalproyectos.reniec.gob.pe:443/PLPI has failed this policy. If a cookie is marked with the "secure" attribute, it will only be transmitted if the communications channel with the host is a secure one. Currently this means that secure cookies will only be sent to HTTPS (HTTP over SSL) servers. If secure is not specified, a cookie is considered safe to be sent in the clear over unsecured channels.

Fix:

Set the `Secure` attribute on the Cookie so that the Cookie is not sent over an unencrypted HTTP request.

Require SSL for an Authentication Cookie (IIS 7):
http://technet.microsoft.com/en-us/library/cc771633(WS.10).aspx

AnonymousIdentificationSection Class [IIS 7]:
http://msdn.microsoft.com/en-us/library/ms689482.aspx

Use the following links to remediate this issue on an Apache server:
http://search.cpan.org/~jkrasnoo/ApacheCookieEncrypted-0.03/Encrypted.pm
http://hc.apache.org/httpclient-3.x/apidocs/org/apache/commons/httpclient/class-use/Cookie.html

Set-Cookie - HTTP MDN (mozilla.org)
https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Set-Cookie#secure

Secure Cookie Attribute OWASP Foundation
https://owasp.org/www-community/controls/SecureCookieAttribute

Reference:

General Information:
The Unofficial Cookie FAQ

File Names:

- https://portalproyectos.reniec.gob.pe:443/PLPI

Medium **Privacy Violation: Shoulder Surfing**

Summary:

Basic web application security measures include “masking” all passwords to ensure they are not easily visible/recovered. This includes both passwords typed in by the user (which may be viewed by someone looking over the user's shoulder), as well as hidden passwords returned by the server (which may be viewed by looking at the HTML source code in the browser). Recommendations include requiring all password fields in your web application be masked to prevent other users from seeing this information.

Fix:

For Security Operations:
Implement a security policy that prevents display of passwords in clear text.

For Development:

Ensure that passwords entered by users on a site are not displayed in clear text. Also, do not return passwords to the user within hidden form fields.

For QA:
Ensure that passwords entered by users on a site are not displayed in clear text by logging it as a defect and notifying the appropriate developer.

File Names:
- https://portalproyectos.reniec.gob.pe:443/PLPI/loginDatosDni

Medium **Cross-Frame Scripting**

Summary:

A Cross-Frame Scripting (XFS) vulnerability can allow an attacker to load the vulnerable application inside an HTML iframe tag on a malicious page. The attacker could use this weakness to devise a Clickjacking attack to conduct phishing, frame sniffing, social engineering or Cross-Site Request Forgery attacks.

Clickjacking
The goal of a Clickjacking attack is to deceive the victim (user) into interacting with UI elements of the attacker's choice on the target web site without their knowledge and then executing privileged functionality on the victim's behalf. To achieve this goal, the attacker must exploit the XFS vulnerability to load the attack target inside an iframe tag, hide it using Cascading Style Sheets (CSS) and overlay the phishing content on the malicious page. By placing the UI elements on the phishing page so they overlap with those on the page targeted in the attack, the attacker can ensure that the victim must interact with the UI elements on the target page not visible to the victim.

WebInspect has detected a response containing one or more forms that accept user input but is missing XFS protection.

An effective frame-busting technique was not observed while loading this page inside a frame.

Execution:

Create a test page containing an HTML iframe tag whose src attribute is set to https://portalproyectos.reniec.gob.pe:443/PLPI/loginDatosDni. Successful framing of the target page indicates that the application is susceptibile to XFS.

Note that WebInspect will report only one instance of this check across each host within the scope of the scan. The other visible pages on the site may, however, be vulnerable to XFS as well and therefore should be protected against it with an appropriate fix.

Implication:

A Cross-Frame Scripting weakness could allow an attacker to embed the vulnerable application inside an iframe. Exploitation of this weakness could result in:

Hijacking of user events such as keystrokes
Theft of sensitive information
Execution of privileged functionality through combination with Cross-Site Request Forgery attacks

Fix:

The Content Security Policy (CSP) frame-ancestors directive obsoletes the X-Frame-Options header. Both provide for a policy-based mitigation technique against cross-frame scripting vulnerabilities. The difference is that while the X-Frame-Options technique only checks against the top-level document's location, the CSP frame-ancestors header checks for conformity from all ancestors.

If both CSP frame-ancestors and X-Frame-Options headers are present and supported, the CSP directive will prevail. WebInspect recommends using both CSP frame-ancestors and X-Frame-Options headers as CSP is not supported by Internet Explorer and many older versions of other browsers.

In addition, developers must also use client-side frame busting JavaScript as a protection against XFS. This will enable users of older browsers that do not support the X-Frame-Options header to also be protected from Clickjacking attacks.

X-Frame-Options
Developers can use this header to instruct the browser about appropriate actions to perform if their site is included inside an iframe.Developers must set the X-Frame-Options header to one of the following permitted values:

- DENY
Deny all attempts to frame the page
- SAMEORIGIN
The page can be framed by another page only if it belongs to the same origin as the page being framed

Content-Security-Policy: frame-ancestors

Developers can use the CSP header with the frame-ancestors directive, which replaces the X-Frame-Options header, to instruct the browser about appropriate actions to perform if their site is included inside an iframe. Developers can set the frame-ancestors attribute to one of the following permitted values:

- 'none'

Equivalent to "DENY" - deny all attempts to frame the page

- 'self'

Equivalent to "SAMEORIGIN" - the page can be framed by another page only if it belongs to the same origin as the page being framed

- <scheme-source>

Developers can also specify a schema such as http: or https: that can frame the page.

Reference:

Frame Busting:
Busting Frame Busting: A Study of Clickjacking Vulnerabilities on Popular Sites
OWASP: Busting Frame Busting

OWASP:
Clickjacking

Content-Security-Policy (CSP)
CSP: frame-ancestors

Specification:
Content Security Policy Level 2
X-Frame-Options IETF Draft

Server Configuration:
IIS
Apache, nginx

HP 2012 Cyber Security Report
The X-Frame-Options header - a failure to launch

File Names:
- https://portalproyectos.reniec.gob.pe:443/PLPI/loginDatosDni

---

Medium **Insecure Deployment: OpenSSL**

Summary:

WebInspect has detected an SSL/TLS man-in-the-middle (MitM) vulnerability caused by a specially crafted SSL handshake message. Also known as OpenSSL ChangeCipherSpec (CCS) injection vulnerability, this bug is known to manifest in the OpenSSL implementation of the secure socket layer for versions earlier than 0.9.8za, 1.0.0 before 1.0.0m and 1.0.1 before 1.0.1h. A ChangeCipherSpec message signals peers to switch to a symmetric encryption during SSL handshake process after negotiating a master key to use for symmetric encryption. However, vulnerable versions of OpenSSL allow a CCS message before the master key is negotiated resulting in a zero length master key.

Execution:

To verify the vulnerability, determine the version of the OpenSSL library deployed on the application servers. The version information can be obtained by running the command openssl version. Note that the client needs to be directly connected to the server in order to test for the vulnerability. If a proxy server is present in the environment, it is recommended to test both the proxy and the application server for the vulnerability.

Implication:

Man-in-the-middle attackers may use this vulnerability to hijack and intercept secure SSL/TLS communication by issuing an early CCS message to client and server when both are using vulnerable instance of OpenSSL. This may allow the attacker to compromise the confidentiality of sensitive session data.

Fix:

Upgrade to latest OpenSSL version.

- OpenSSL 0.9.8 SSL/TLS users should upgrade to 0.9.8za or later.
- OpenSSL 1.0.0 SSL/TLS users should upgrade to 1.0.0m or later.
- OpenSSL 1.0.1 SSL/TLS users should upgrade to 1.0.1h or later.

Additionally, users should check for the possibility of an indirect dependency on the OpenSSL library via third party software. Upgrade any such instances according to vendor recommendations.

Reference:

- https://www.openssl.org/news/secadv_20140605.txt
- http://web.nvd.nist.gov/view/vuln/detail?vulnId=CVE-2014-0224

File Names:
- https://portalproyectos.reniec.gob.pe:443/PLPI

Medium **Insecure Deployment: Path Normalization Conflict**

Summary:

WebInspect has detected a Path Normalization Inconsistency Problem in a multi-layered architecture. Each URL Path may have several alternative URL representations. The goal of the normalization process is to transform a URL into a form so that all syntactical variants of the same semantic URL will be treated equally. In multi-layered architectures where the client

request is routed through various intermediate components such as a load-balancer, a reverse proxy, or a web application firewall there is a risk that normalization is different at each layer. In this case, an attacker may be able to bypass allow list and deny list ACL rules or escape from context mapping. As a result, sensitive resources from the server will be unexpectedly exposed.

Example: The following configuration of NGINX forwards all requests for */publicfolder*to the Tomcat server:

```
location /publicfolder/ {
    proxy_pass http://TomcatServer:8080/;
    }
 ...
```

In the example above, it is expected that only */publicfolder*from the Tomcat server is accessible from outside.However, an attacker can send the next request: `http://targetserver/publicfolder/..;/manager/html`NGINX considers `..;` as a folder name, so the path matches the NGINX rule and is forwarded to the Tomcat server. The Tomcat server considers `..;` as a folder with a path parameter and will map the request to */publicfolder/../manager/htm*l, resulting in the attacker gaining access to the manager app on the Tomcat server.

Execution:

Open a web browser and navigate to https://portalproyectos.reniec.gob.pe:443/PLPI/..;/PLPI/loginDatosDni.If the server responds with 200 OK, try to change the name of the last directory before `..;/` in the path to a non-existent directory and send the request to the server. If the server responds with 404 File Not Found error, the configuration has a Path Normalization Inconsistency.

Implication:

Path Normalization Inconsistency Problem can allow an attacker to bypass allow list, and/or deny list, ACL. Alternatively, an attacker may escape from the context mapping to gain access to sensitive resources that would otherwise be inaccessible.

Fix:

- Refer to the vendor advisory for the intermediate component and the web server. Replace the component with a version or product that will have a Path Normalization process identical to the other components and will avoid Path Normalization Inconsistency.
- Consider removing unnecessary sensitive resources from Web server.
- Add additional security protection on the Web Server such as forbidding requests from intermediate components to sensitive resources like server management console, etc. and/or add authentication to such sensitive resources.

Reference:

Breaking Parser Logic by Orange Tsai

File Names:

- https://portalproyectos.reniec.gob.pe:443/PLPI/..;/PLPI/loginDatosDni

---

Low

Web Server Misconfiguration: Unprotected Directory

Summary:

Directory Enumeration vulnerabilities were discovered within your web application. Risks associated with an attacker discovering a directory on your application server depend upon what type of directory is discovered, and what types of files are contained within it. The primary threat, other than accessing files containing sensitive information, is that an attacker can utilize the information discovered in that directory to perform other types of attacks. Recommendations include restricting access to important directories or files by adopting a "need to know" requirement for both the document and server root, and turning off features such as Automatic Directory Listings that provide information that could be utilized by an attacker when formulating or conducting an attack.

Fix:

For Security Operations:
You should evaluate the production requirements for the found directory. If the directory is not required for production operation, then the directory and its contents should be removed or restricted by a server access control mechanism. More information about implementing access control schemes can be found in the References. Automatic directory indexing should also be disabled, if applicable.

For Development:
This problem will be resolved by the web application server administrator. In general, do not rely on 'hidden' directories within the web root that can contain sensitive resources or web applications. Assume an attacker knows about the existence of all directories and files on your web site, and protect them with proper access controls.

For QA:
This problem will be resolved by the web application server administrator.

Reference:

Implementing Basic Authentication in IIS

http://www.microsoft.com/technet/prodtechnol/WindowsServer2003/Library/IIS/abbca505-6f63-4267-aac1-1ea89d861eb4.mspx

Implementing Basic Authentication in Apache
http://httpd.apache.org/docs/howto/auth.html#intro

File Names:

- https://portalproyectos.reniec.gob.pe:443/cgi-bin/

Low

Web Server Misconfiguration: Server Error Message

Summary:

A server error response was detected. The server could be experiencing errors due to a misbehaving application, a misconfiguration, or a malicious value sent during the auditing process. While error responses in and of themselves are not dangerous, per se, the error responses give attackers insight into how the application handles error conditions. Errors that can be remotely triggered by an attacker can also potentially lead to a denial of service attack or other more severe vulnerability. Recommendations include designing and adding consistent error handling mechanisms which are capable of handling any user input to your web application, providing meaningful detail to end-users, and preventing error messages that might provide information useful to an attacker from being displayed.

Implication:

The server has issued a 500 error response. While the body content of the error page may not expose any information about the technical error, the fact that an error occurred is confirmed by the 500 status code. Knowing whether certain inputs trigger a server error can aid or inform an attacker of potential vulnerabilities.

Fix:

For Security Operations:

Server error messages, such as "File Protected Against Access", often reveal more information than intended. For instance, an attacker who receives this message can be relatively certain that file exists, which might give him the information he needs to pursue other leads, or to perform an actual exploit. The following recommendations will help to ensure that a potential attacker is not deriving valuable information from any server error message that is presented.

- Uniform Error Codes: Ensure that you are not inadvertently supplying information to an attacker via the use of inconsistent or "conflicting" error messages. For instance, don't reveal unintended information by utilizing error messages such as Access Denied, which will also let an attacker know that the file he seeks actually exists. Have consistent terminology for files and folders that do exist, do not exist, and which have read access denied.
- Informational Error Messages: Ensure that error messages do not reveal too much information. Complete or partial paths, variable and file names, row and column names in tables, and specific database errors should never be revealed to the end user. Remember, an attacker will gather as much information as possible, and then add pieces of seemingly innocuous information together to craft a method of attack.
- Proper Error Handling: Utilize generic error pages and error handling logic to inform end users of potential problems. Do not provide system information or other data that could be utilized by an attacker when orchestrating an attack.

Removing Detailed Error Messages

Find instructions for turning off detailed error messaging in IIS at this link:

http://support.microsoft.com/kb/294807

For Development:

From a development perspective, the best method of preventing problems from arising from server error messages is to adopt secure programming techniques that prevent problems that might arise from an attacker discovering too much information about the architecture and design of your web application. The following recommendations can be used as a basis for that.

- Stringently define the data type (for instance, a string, an alphanumeric character, etc) that the application will accept.
- Use what is good instead of what is bad. Validate input for improper characters.
- Do not display error messages to the end user that provide information (such as table names) that could be utilized in orchestrating an attack.
- Define the allowed set of characters. For instance, if a field is to receive a number, only let that field accept numbers.
- Define the maximum and minimum data lengths for what the application will accept.
- Specify acceptable numeric ranges for input.

For QA:

The best course of action for QA associates to take is to ensure that the error handling scheme is consistent. Do you receive a different type of error for a file that does not exist as opposed to a file that does? Are phrases like "Permission Denied" utilized which could reveal the existence of a file to an attacker? Inconsistent methods of dealing with errors gives an attacker a very powerful way of gathering information about your web application.

Reference:

Apache:
Security Tips for Server Configuration
Protecting Confidential Documents at Your Site
Securing Apache - Access Control

Microsoft:
How to set required NTFS permissions and user rights for an IIS 5.0 Web server
Default permissions and user rights for IIS 6.0
Description of Microsoft Internet Information Services (IIS) 5.0 and 6.0 status codes

File Names:

- https://portalproyectos.reniec.gob.pe:443/<script>alert('TRACK');</script>
- https://portalproyectos.reniec.gob.pe:443/PLPI/usr/share/zoneinfo/GMT%3ffakestp.html
- https://portalproyectos.reniec.gob.pe:443/PLPI/consultaPadron/cargar/.%3Fzxz%3F
- https://portalproyectos.reniec.gob.pe:443/PLPI/resources1/js/.%3Fzxz%3F.jquery-3.3.1.min.js
- https://portalproyectos.reniec.gob.pe:443/PLPI/.%3Fzxz%3F.loginDatosDni
- https://portalproyectos.reniec.gob.pe:443/PLPI/GET
- https://portalproyectos.reniec.gob.pe:443/cgi-bin/README.txt
- https://portalproyectos.reniec.gob.pe:443/PLPI/consultaPadron/buscarDni/.%3Fzxz%3F
- https://portalproyectos.reniec.gob.pe:443/PLPI/consultaPadron/cargar/
- https://portalproyectos.reniec.gob.pe:443/GET
- https://portalproyectos.reniec.gob.pe:443/PLPI/consultaPadron/ordenar/.%3Fzxz%3F
- https://portalproyectos.reniec.gob.pe:443/usr/share/zoneinfo/GMT%3ffakestp.html
- https://portalproyectos.reniec.gob.pe:443/PLPI/consultaPadron/guardarAlerta/.%3Fzxz%3F

Low **Setting Manipulation: Character Set**

Summary:

A vulnerability was detected in your web application that allows a user to control the HTML character encoding used to parse the HTTP response of a given request. Attackers can exploit this vulnerability to evade certain validation mechanisms used for Cross-site Scripting.
The response character encoding is used by a web browser to decide how to interpret the characters in the body of the HTTP response. The most common encoding used by web applications today is UTF-8. The character set (charset) declaration is usually done through a header in the HTTP response or using the HTML <meta> tag. Such declarations should be controlled by the application only. If this declaration is controlled through user input, then an attacker can use this feature to modify the charset that will be used by the browser and modify the interpretation of the contents of the response. This can allow for Cross-site Scripting attacks that would otherwise not have succeeded while using UTF-8 encoding.

Execution:

This vulnerability is detected by modifying an input parameter value in an HTTP request to a different charset. The headers and the HTML <meta> tag of the corresponding HTTP response are then observed to confirm a successful manipulation of the response charset.

Implication:

A successful exploitation of this vulnerability could increase the probability of a successful Cross-site Scripting attack by evading existing server-side input validation routines.
For example:

```
+ADw-script+AD4-alert(document.location)+ADw-/script+AD4
```

The above string means nothing in most encoding types, and therefore is "safe", but when a victim views this under utf-7 encoding, it will be interpreted as valid html tag and hence, the script will be executed.

Fix:

The ideal solution would be to control charset declarations from the application and never through user-supplied input. If a particular feature of the application demands such a capability, then it is advisable to use an allow list of allowed charsets and validate that proper input validation routines are in place for all the values in the allow list.

Reference:

Browser Security Handbook
Computer Security Research - Secunia
Maia Mailguard 'charset' Parameter HTML Injection Vulnerability
OWASP Encoding Project

File Names:

- https://portalproyectos.reniec.gob.pe:443/PLPI/sendLogin
- https://portalproyectos.reniec.gob.pe:443/PLPI/sendLogin
- https://portalproyectos.reniec.gob.pe:443/PLPI/sendLogin
- https://portalproyectos.reniec.gob.pe:443/PLPI/sendLogin

Low

Cache Management: Insecure Policy

Summary:

WebInspect has detected a potentially unsafe cache control policy for secure content. While content transmitted over an SSL/TLS channel is expected to guarantee confidentiality, administrators must ensure that caching of sensitive content is disabled unless absolutely needed. The misconception that secure content caching is disabled by default by user-agents could cause the application to fail the organization's cache policy. An unsafe specification such as Cache-Control: public instructs the browser to persistently cache the content on the hard drive. Cache-Control with no-store in the value must be set to prevent browsers from persisting content. Browsers and intermediate proxies will still persists with the no-cache directive. However, they will revalidate the content with the server before serving content from cache. The private directive prevents intermediate proxies from caching content. Missing Cache-Control policy header results in browsers caching content regardless of whether it is served over HTTP or HTTPS.

- Cache-Control: no-store

Execution:

Send a request to https://portalproyectos.reniec.gob.pe:443/PLPI and inspect the Cache-Control header value.

Implication:

Insecure caching policies could lead to content spoofing or information theft.

SSL provides secure encrypted channel to transfer information from source to user. The information served over SSL is considered sensitive and trusted to be only available to the requestor. However, caching this content on disk in temporary internet files or on an intermediate proxy server can compromise that trust by exposing it to anyone who has access to the temporary storage or proxy cache. Content served over SSL should have cache disabled.

Fix:

Set the Cache-Control directive to no-store.

no-cache
For sensitive resources requiring user authentication, servers can send the no-cache directive to prevent caches from serving a cached response without first requiring the user-agent to validate the user identity. This directive can be specified with or without field names. When no field names are included, this directive applies to the entire request or response.
When one or more field names are specified in the no-cache directive, the response is cached but the specified fields must be excluded. If the response must include the specified field, then the cache must ensure that the request triggers a revalidation with the origin server.
Example: Cache-Control: no-cache="Set-Cookie"
This directive can be used to prevent sensitive information leakage by requiring the server to confirm the user identity before serving the protected information. While content is not served using the browser's history or back button, a user with disk access can retrieve the content.

no-store
To completely disable the caching of requests or responses, the server must specify the no-store directive in the Cache-Control header. This directive applies to the entire request and response regardless of whether the directive is sent in the request or the response.

Reference:

Server Configuration:
IIS
http://support.microsoft.com/kb/247404
Apache
https://httpd.apache.org/docs/2.4/caching.html

RFC 9111 Section 5.2 Cache Control
https://datatracker.ietf.org/doc/html/rfc9111#section-5.2

Browser Cache FAQ
https://www.owasp.org/index.php/OWASP_Application_Security_FAQ#Browser_Cache

Caching Tutorial
https://www.mnot.net/cache_docs/

File Names: • https://portalproyectos.reniec.gob.pe:443/PLPI

Low **Insecure Transport: HSTS not Set**

Summary:

Http Strict Transport Security (HSTS) policy enables web applications to enforce web browsers to restrict communication with the server over an encrypted SSL/TLS connection for a set period. Policy is declared via special Strict Transport Security response header. Encrypted connection protects sensitive user and session data from attackers eavesdropping on network connection.
Consider following attack scenarios:

- Users often omit the URI scheme i.e. https:// when typing a URL in location bar to access a website. Also third party websites can link to the site using the "http" scheme instead of ""https". This could result in an initial connection to a HTTPS-enabled site over an unencrypted channel. An eavesdropping attacker can hijack this unencrypted connection and replace the intended use of HTTPS protocol with HTTP in an attack known as SSLStrip, granting unauthorized access to all subsequent traffic.
- Websites often transfer non-sensitive resources such as help documents over an unencrypted HTTP connection. Any cookies without a secure flag are sent along with such requests potentially disclosing sensitive user and session data to eavesdropper.
- Man-in-the-Middle attacks that exploit user tendencies to override invalid certification warnings, e.g. SSLSniff.

For web sites configured with an accurate HSTS policy, browsers automatically upgrade any HTTP connections to HTTPS. Furthermore, browsers prevent users from overriding any host certificate warnings. HSTS offers an effective defense against above attack scenarios.

Execution:

Access location https://portalproyectos.reniec.gob.pe:443/ and notice the absence of the Strict Transport Security header in the HTTP response.

Implication:

A successful MiTM attack such as SSLStrip or SSLsniff can lead to the compromise of sensitive user data such as financial information, Social Security Number, personal information etc. as well as grant unauthorized access to user accounts enabling attackers to perform privileged actions on client's behalf.

Fix:

Configure the web application under test to include Strict Transport Security header in every response generated by an HTTPS -enabled site. Any HTTP version of site on the same domain should permanently redirect to the secure encrypted site. Header should not be added to HTTP response as browsers will ignore it.

It is important to note that this header does not prevent from above mentioned attack scenarios during the very first connection to the site or any connections established after the set period has expired. To prevent such a scenario, the site must be added to the pre-loaded HSTS hosts list embedded in both Google Chrome and Mozilla Firefox browsers.

Reference:

http://tools.ietf.org/html/rfc6797

File Names: • https://portalproyectos.reniec.gob.pe:443/

Low **HTML5: Misconfigured Content Security Policy**

Summary:

Content Security Policy (CSP) is a declarative security header that allows developers to dictate which domains the site is allowed to load contents from or initiate connection to when rendered in the web browser. It provides an additional layer of security from critical vulnerabilities like cross site scripting, clickjacking, cross origin access etc. on top of input validation and having an allow list in code. Improperly configured header however, fails to provide this additional layer of security. The policy is defined with the help of fifteen directives including eight that control resource access namely -

- script-src
- img-src
- object-src
- style_src
- font-src
- media-src
- frame-src
- connect-src

Each of these takes a source list as a value specifying domains the site is allowed to access for feature covered by that directive. Developers may use wildcard * to indicate all or part of the source. None of the directives are mandatory. Browsers will either allow all sources for unlisted directive or will derive its value from optional default-src directive. Furthermore, the specification for this header has evolved over time. It was implemented as X-Content-Security-Policy in Firefox until version 23 and in IE until version 10, and was implemented as X-Webkit-CSP in Chrome until version 25. Both of the names are deprecated in favor of the now standard name Content Security Policy. Given number of directives, two deprecated alternate names, and the way multiple occurrences of same header and repeat directive in single header is treated there is a high

propensity that developer might misconfigure this header.

Consider following misconfiguration scenarios:

- It can be an overly permissive policy if default-src is not set or set to wildcard and/or other directives are set to wildcard.
- Multiple instances of this header are allowed in same response. A development team and security team might both set the header but with either different policies or one may use now deprecated X-Content-Security-Poilcy or X-Webkit-CSP headers. Deprecated versions are ignored if standard Content-Security-Policy header is present.
- If a directive is repeated within the same instance of the header, all subsequent occurrences are ignored.

Following misconfigurations were detected at https://portalproyectos.reniec.gob.pe:443/PLPI:

Overly Permissive Content Security Policy.

- *Content-Security-Policy:default-src,script-src,object-src,font-src,style-src,img-src,frame-src,connect-src,media-src* are set to wildcard.

Execution:

Access link https://portalproyectos.reniec.gob.pe:443/PLPI through proxy and notice the misconfigured content security policy header in response headers.

Implication:

Content security policy header provides additional layer of security for site from client side attacks such as cross-site scripting. In absence of this header, an attacker can exploit client side vulnerabilities such as cross-site scripting, clickjacking and cross-site request forgery.

Fix:

Remove wildcard values for source-list to limit scope of cross origin access from site. Ensure canonical *Content Security Policy* name is used to specify policy. X-Content-Security-Policy and X-Webkit-CSP names are deprecated. Any references to these headers are only useful in case support for earlier browsers is desired. Presence of these headers in addition to Content-Security-Policy is said to cause unexpected behaviors on certain versions of browsers.
Content-Security-Policy version 2 standard support:

- Edge: Edge 15 - 18 supported with a nonce bug. Version 75 onwards fully supported.
- Chrome: Chrome 36-38 are missing the plugin-types, child-src, frame-ancestors, base-uri, and form-action directives. Chrome 39 is missing the plugin-types, child-src, base-uri, and form-action directives. Chrome 40 onwards fully supported.
- Firefox; Firefox 31-34 is missing the plugin-types, child-src, frame-ancestors, base-uri, and form-action directives. Firefox 35 is missing the plugin-types, child-src, frame-ancestors, and form-action directives. Firefox 36-44 is missing the plugin-types and child-src directives. Firefox 45+ is missing the plugin-types directive.

Furthermore, the report-uri directive can be configured to receive reports of attempts to violate the policy. These reports can be used as an early indication of security issues in the site as well as to optimize the policy.

Reference:

http://www.w3.org/TR/CSP/
https://owasp.org/www-community/controls/Content_Security_Policy
https://developer.mozilla.org/en-US/docs/Web/Security/CSP/Introducing_Content_Security_Policy
https://content-security-policy.com

File Names:

- https://portalproyectos.reniec.gob.pe:443/PLPI

Low **Cookie Security: Missing SameSite Attribute**

Summary:

The SameSite attribute protects cookies from Cross-Site Request Forgery (CSRF) attacks. The browser automatically appends cookies to every HTTP request made to the site that sets the cookie. Cookies might store sensitive data like session ID and authorization token or site data that is shared between different requests to the same site during a session. An attacker can perform an impersonation attack by generating a request to the authenticated site from a third-party site page loaded on the client machine because the browser automatically appended the cookie to the request.
The SameSite attribute on a cookie allows sites to control that behaviour and prevents browsers from appending the cookie to request if the request is generated from a third-party site page load. The SameSite attribute can have the following three values:

- Strict: When set to Strict, cookies are only sent along with requests upon top level navigation .
- Lax: When set to Lax, cookies are sent with top level navigation from the same host as well as GET requests originated to the host from third-party sites (for example, in iframe, link, href, and so on and the form tag with GET method only).

- None: Cookies are sent in all requests made to the site within the path and domain scope set for the cookie. Requests generated due to form submissions using the POST method are also allowed to send cookies with request.

Please note that cookies that have the SameSite attribute with the value of None must be set with the Secure attribute otherwise the browser rejects the cookies. Additionally, a few specific browser versions reject the SameSite cookie with the None value for example, Chrome versions 51 to 66, versions of the UC Browser on Android prior to version 12.13.2, versions of Safari and embedded browsers on macOS 10.14, and all browsers on iOS 12 reject cookies set with SameSite=None. A suggested workaround for this issue is to set an alternate cookie with a prefix or suffix such as *Legacy*appended to cookiename . Sites can look for this legacy cookie if it does not find a cookie that was set with SameSite=None.

Execution:

Inspect the highlighted cookie value in the HTTP response in the vulnerable session. The cookie is missing the SameSite attribute.

Implication:

Sites that set cookies without the SameSite attribute have an increased risk of CSRF attacks. Using CSRF, an attacker can impersonate a valid user and gain unauthorized access to application functionality. Furthermore, the recent versions of browsers might reject the cookie that is not set with the SameSite attribute.

Fix:

Add the SameSite attribute to all cookies. Starting February 2020, the Chrome browser made SameSite a mandatory attribute for all cookies. Any cookie without the SameSite attribute is either rejected or treated with the default behaviour, which is the same as setting the attribute value to Lax. Therefore, any cookie that must be sent regardless the origination of the request (for example, analytics cookies) should have the SameSite attribute value of none.
Furthermore, we recommend that developers continue to add traditional CSRF mitigations to the site along with SameSite attribute. Many site users might still use older browser versions to access the site. Older browser versions do not understand the SameSite attribute.

Reference:

https://tools.ietf.org/html/draft-ietf-httpbis-rfc6265bis-05
https://www.chromium.org/updates/same-site/incompatible-clients
https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Set-Cookie/SameSite

File Names:
- https://portalproyectos.reniec.gob.pe:443/PLPI

Low **Cache Management: Web Cache Poisoning**

Summary:

WebInspect has detected a web cache poisoning instance by fat GET requests. Fat GET requests are requests that contain parameters as request body. Web cache poisoning exploits the behaviour of the web server causing a malicious response to be served to legitimate requests. In this type of web cache poisoning, the cache key is derived from the original query parameters passed in the request line. However, the value of the query parameter used by the server is the duplicate value sent as part of the request body. This altered response is cached and served to the users with legitimate requests.

WebInspect has detected that the cache can be poisoned. The **lg** was passed in the request body and it is observed that the initial response differed from the later response for the same query. This indicates that the cache might be poisoned.

Execution:

To test for web cache poisoning, execute the following steps:

The cache-buster is a query parameter added to a URL for example, `cacheBuster=someRandomValue`. Add a cache-buster to the URL.
Add the query parameter as part of the request body for example `param=someRandomValue`.
Send the request and wait for the response.
Next, remove the request body and send the request.
The response received is the malicious cached response.

Implication:

If the web server is susceptible to web cache poisoning, it can lead to legitimate users being served malicious responses.

Fix:

FAT GET requests must not be accepted. Either prune the request body or do not accept such a request.

Reference:

Web Cache Poisoning

File Names:
- https://portalproyectos.reniec.gob.pe:443/PLPI/login?lg=1&cacheBuster=WEBINSPECT1418034381

Low **HTML5: Deprecated Header**

Summary:

`X-XSS-Protection` header is enabled. `X-XSS-Protection` refers to a header used by developers to control the browser's behavior and offers protection against cross-site scripting (XSS). The header was designed to filter pages to remove unsafe

components or stop loading when cross-site scripting was detected. Internet Explorer 8 introduced the `X-XSS-Protection` header and it was also supported in Edge version 12-16, Chrome version 4-77, Opera version 15-64, and Safari 5-15.3. Modern browsers no longer maintain, support, or enhance implementation of this header.

Having this setting enabled does not necessarily result in a vulnerability, but in some cases such filters are leveraged to perform XSS attacks against users that would otherwise not be possible. The XSS filters in the browser use regular expressions to alter the response. For example, a regex to detect `<script>` tag would alter the tag in the response to `<sc#ipt>`. It would stop the current XSS attack but also have unintended consequences. An attacker could use this knowledge to disable legitimate scripts or client-side scripts containing security features by adding a malicious parameter. By setting the `X-XSS-Protection` to `"1; mode=block"` stops the entire page from loading but it is vulnerable to side-channel attacks [3].

This header can be set explicitly in an application server configuration or in application code. All places should be checked for insecure setting of this header. This check is reported once per host by default and can be made to report all instances by setting check input `"X-XSS-Protection_Aggressive Reporting"`

Execution:

Click the response tab of the highlighted request. You will notice the occurrence of `X-XSS-Protection` header highlighted and value is set to 1.

Implication:

Setting `X-XSS-Protection` header is ineffective for browsers that no longer support this header. Enabling `X-XSS-Protection` header might increase the risk of cross-site scripting attacks against the application if a user accesses it from an older browser that supported this feature.

Fix:

Remove the response header `X-XSS-Protection`.
OpenText recommends using `Content Security Policy (CSP)` to protect against XSS attacks.

Reference:

1 X-XSS-Protection - HTTP MDN
https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-XSS-Protection

2 IE8 XSS Filters
https://media.blackhat.com/bh-eu-10/presentations/Lindsay_Nava/BlackHat-EU-2010-Lindsay-Nava-IE8-XSS-Filters-slides.pdf

3 Abusing Chrome's XSS auditor to steal tokens PortSwigger Research
https://portswigger.net/research/abusing-chromes-xss-auditor-to-steal-tokens

File Names:

- https://portalproyectos.reniec.gob.pe:443/PLPI

Informational **System Information Leak: External**

Summary:

A URL or filename was found in the comments of the file.

File Names:

- https://portalproyectos.reniec.gob.pe:443/PLPI/inicio
- https://portalproyectos.reniec.gob.pe:443/PLPI/sendLogin
- https://portalproyectos.reniec.gob.pe:443/PLPI/sendLogin
- https://portalproyectos.reniec.gob.pe:443/PLPI/login?lg=1
- https://portalproyectos.reniec.gob.pe:443/PLPI/sendLogin
- https://portalproyectos.reniec.gob.pe:443/PLPI/login?lg=1
- https://portalproyectos.reniec.gob.pe:443/PLPI/inicio
- https://portalproyectos.reniec.gob.pe:443/PLPI/inicio
- https://portalproyectos.reniec.gob.pe:443/PLPI/inicio
- https://portalproyectos.reniec.gob.pe:443/PLPI/sendLogin
- https://portalproyectos.reniec.gob.pe:443/PLPI/sendLogin
- https://portalproyectos.reniec.gob.pe:443/PLPI/sendLogin
- https://portalproyectos.reniec.gob.pe:443/PLPI/inicio
- https://portalproyectos.reniec.gob.pe:443/PLPI/login?lg=1
- https://portalproyectos.reniec.gob.pe:443/PLPI/login?lg=1

Informational **Hidden Field**

Summary:

While preventing display of information on the web page itself, the information submitted via hidden form fields is easily accessible, and could give an attacker valuable information that would prove helpful in escalating his attack methodology.

Recommendations include not relying on hidden form fields as a security solution for any area of the web application that contains sensitive information or access to privileged functionality such as remote site administration functionality.

Execution:

Any attacker could bypass a hidden form field security solution by viewing the source code of that particular page.

Implication:

The greatest danger from exploitation of a hidden form field design vulnerability is that the attacker will gain information that will help in orchestrating a far more dangerous attack.

Fix:

Do not rely on hidden form fields as a method of passing sensitive information or maintaining session state. One workable bypass is to encrypt the hidden values in a form, and then decrypt them when that information is to be utilized by a database operation or a script. From a security standpoint, the best method of temporarily storing information required by different forms is to utilize a session cookie.

Whether hidden or not, if your site utilizes values submitted via a form to construct database queries, do not make the assumption that the data is non-malicious. Instead, utilize the following recommendations to sanitize user supplied input.

- Stringently define the data type (for instance, a string, an alphanumeric character, etc) that the application will accept.
- Use what is good instead of what is bad.
- Validate input for improper characters.
- Do not display error messages to the end user that provide information (such as table names) that could be utilized in orchestrating an attack.
- Define the allowed set of characters. For instance, if a field is to receive a number, only let that field accept numbers.
- Define the maximum and minimum data lengths for what the application will accept.
- Specify acceptable numeric ranges for input.

File Names:

- https://portalproyectos.reniec.gob.pe:443/PLPI/loginDatosDni
- https://portalproyectos.reniec.gob.pe:443/PLPI
- https://portalproyectos.reniec.gob.pe:443/PLPI
- https://portalproyectos.reniec.gob.pe:443/PLPI/login?lg=1
- https://portalproyectos.reniec.gob.pe:443/PLPI/sendLogin
- https://portalproyectos.reniec.gob.pe:443/PLPI/inicio

Informational

Web Server Misconfiguration: OPTIONS HTTP Method

Summary:

The server supports the OPTIONS HTTP method. The OPTIONS method is used to determine what other methods the server supports for a given URI/resource.

Reference:

RFC 2616 Section 9: HTTP Methods:
http://www.w3.org/Protocols/rfc2616/rfc2616-sec9.html

Apache:
Apache HTTP Server Version 2.0
Apache HTTP Server Version 1.3

Microsoft:
UrlScan Security Tool
How to configure the URLScan Tool
Setting Application Mappings in IIS 6.0

File Names:
- https://portalproyectos.reniec.gob.pe:443/

Informational **HTML5: Missing Content Security Policy**

Summary:

Content Security Policy (CSP) is an HTTP response security header that developers and security architects can leverage to create an allow list of domains from which the site is allowed to load resources. This header provides an in-depth security protection from critical vulnerabilities such as cross-site scripting and clickjacking. Additionally, CSP restricts execution of inline JavaScript, dynamic JavaScript code evaluation from strings, and framing of the site from external domains. While CSP is not a replacement for input validation, it can help to significantly reduce the risk of XSS from unknown weaknesses. The CSP frame-ancestors directive is equivalent to X-Frame-Options and restricts the domain that are allowed to frame the site's content.

Execution:

Access link https://portalproyectos.reniec.gob.pe:443/GET through a proxy and notice the missing CSP header in the response. By default, WebInspect flags only one instance of this vulnerability per host because it is typical to set this header at the host level in a server configuration.

Perform the following steps to flag all instances of this issue:

- Create a new policy with the selection of checks that you want to include in a rescan. We recommend using the Blank or Passive policy as a base.
- Select this check and uncheck the "FlagAtHost" check input from standard description.
- Save the policy.
- Rescan with this new custom policy.

Implication:

Security architects and developers can leverage CSP to significantly reduce the risk of XSS and clickjacking attacks. CSP headers can restrict leakage of information to external domains by restricting which domains the site is allowed to load contents from when rendered in browser .

Fix:

Define a CSP policy suitable for your site. The policy can be set either with an HTTP response header or <meta /> tag.

For example:
```
Content-Security-Policy: default-src https://example.net; child-src 'none';
```
Or
```
<meta http-equiv="Content-Security-Policy" content="default-src https://cdn.example.net;
child-src 'none'; object-src 'none'">
```

Content-Security-Policy 2 is the recommended standard. Content-Security-Policy 3 is in draft. The following is a snapshot of modern browser support for the CSP header:

- Edge: Versions 15-18; supported with a nonce bug. Version 75 and later; fully supported.
- Chrome: Versions 36-38; missing the plugin-types, child-src, frame-ancestors, base-uri, and form-action directives. Version 39; missing the plugin-types, child-src, base-uri, and form-action directives. Version 40 and later; fully supported.
- Firefox: Versions 31-34; missing the plugin-types, child-src, frame-ancestors, base-uri, and form-action directives. Version 35; missing the plugin-types, child-src, frame-ancestors, and form-action directives. Versions 36-44; missing the plugin-types and child-src directives. Version 45 and later; missing the plugin-types directive.

Furthermore, the report-uri directive can be configured to receive reports of attempts to violate the policy. These reports can be used as an early indication of security issues in the site as well as to optimize the policy.

Reference:

Content Security Policy Level 3
OWASP Content Security Policy
MDN web docs
Content Security Policy (CSP) Quick Reference Guide

File Names:
- https://portalproyectos.reniec.gob.pe:443/GET

Informational **HLI: Detected Libraries**

Summary:

Hacker Level Insights provides developers and security professionals with more context relating to the overall security posture of their application. The **jQuery UI** version **1.11.4** was detected to be in use by during this scan. While these findings do not necessarily represent a security vulnerability, it is important to note that attackers commonly perform reconnaissance of their target in an attempt to identify known weaknesses or patterns. Knowing what the hacker can see provides context which can help teams better secure their applications.
.cvc-grid {width:100%;}.cvc-box {border-radius: 15px; border: 2px solid #1a75ff; width: 100%; padding: 10px;}.cvc-cat {position: relative; display: inline-block; border-bottom: 1px dotted black;}.cvc-cat .nvd-tooltip {visibility: hidden; width:

400px; background-color: #3b96ff; color: #fff; text-align: left; border-radius: 6px; padding: 5px; position: absolute; z-index: 1;}.cvc-cat:hover .nvd-tooltip {visibility: visible;}.score-box{color:#000!important;background-color:#f1f1f1!important;width:75%;border-radius:8px}.score-box div {border-radius:8px}.score-box div div{padding-left: 10px; font-weight: bold;}.score-box > div:after,.score-box > div:before {content:""; display:table;clear:both}.sb-good{color:#fff!important;background-color:#87e547!important}.sb-warn{color:#fff!important;background-color:#fbc02d!important}.sb-bad{color:#fff!important;background-color:#ff8686!important}.nvd-tooltip:before{content:"The National Vulnerability Database (NVD) helps Fortify WebInspect HLI analysis to find information for CVEs including a brief summary by using Common Platform Enumeration(CPE) queries. The database is maintained by the National Institute of Standards and Technology (NIST)."}

Known NVD records for: cpe:2.3:a:jqueryui:jquery_ui:1.11.4

VULNERABILITY:	FOUND VERSION:	FIXED VERSION:	SEVERITY:
CVE-2016-7103	1.11.4	After: 1.11.4	Medium

Cross-site scripting (XSS) vulnerability in jQuery UI before 1.12.0 might allow remote attackers to inject arbitrary web script or HTML via the closeText parameter of the dialog function.

VULNERABILITY:	FOUND VERSION:	FIXED VERSION:	SEVERITY:
CVE-2021-41182	1.11.4	1.13.0	Medium

jQuery-UI is the official jQuery user interface library. Prior to version 1.13.0, accepting the value of the `altField` option of the Datepicker widget from untrusted sources may execute untrusted code. The issue is fixed in jQuery UI 1.13.0. Any string value passed to the `altField` option is now treated as a CSS selector. A workaround is to not accept the value of the `altField` option from untrusted sources.

VULNERABILITY:	FOUND VERSION:	FIXED VERSION:	SEVERITY:
CVE-2021-41183	1.11.4	1.13.0	Medium

jQuery-UI is the official jQuery user interface library. Prior to version 1.13.0, accepting the value of various `*Text` options of the Datepicker widget from untrusted sources may execute untrusted code. The issue is fixed in jQuery UI 1.13.0. The values passed to various `*Text` options are now always treated as pure text, not HTML. A workaround is to not accept the value of the `*Text` options from untrusted sources.

VULNERABILITY:	FOUND VERSION:	FIXED VERSION:	SEVERITY:
CVE-2021-41184	1.11.4	1.13.0	Medium

jQuery-UI is the official jQuery user interface library. Prior to version 1.13.0, accepting the value of the `of` option of the `.position()` util from untrusted sources may execute untrusted code. The issue is fixed in jQuery UI 1.13.0. Any string value passed to the `of` option is now treated as a CSS selector. A workaround is to not accept the value of the `of` option from untrusted sources.

VULNERABILITY:	FOUND VERSION:	FIXED VERSION:	SEVERITY:
CVE-2022-31160	1.11.4	1.13.2	Medium

jQuery UI is a curated set of user interface interactions, effects, widgets, and themes built on top of jQuery. Versions prior to 1.13.2 are potentially vulnerable to cross-site scripting. Initializing a checkboxradio widget on an input enclosed within a label makes that parent label contents considered as the input label. Calling `.checkboxradio( "refresh" )` on such a widget and the initial HTML contained encoded HTML entities will make them erroneously get decoded. This can lead to potentially executing JavaScript code. The bug has been patched in jQuery UI 1.13.2. To remediate the issue, someone who can change the initial HTML can wrap all the non-input contents of the `label` in a `span`.

File Names:

- https://portalproyectos.reniec.gob.pe:443/PLPI/inicio

- https://portalproyectos.reniec.gob.pe:443/PLPI/inicio
- https://portalproyectos.reniec.gob.pe:443/PLPI/inicio

Best Practice **Compliance Failure: Missing Privacy Policy**

Summary:

A privacy policy was not supplied by the web application within the scope of this audit. Many legislative initiatives require that organizations place a publicly accessible document within their web application that defines their website's privacy policy. As a general rule, these privacy policies must detail what information an organization collects, the purpose for collecting it, potential avenues of disclosure, and methods for addressing potential grievances.
Various laws governing privacy policies include the Gramm-Leach-Bliley Act, Health Insurance Portability and Accountability Act (HIPAA), the California Online Privacy Protection Act of 2003, European Union's Data Protection Directive and others.

Execution:

All of the web pages accessible within the scope of the scan are sampled for textual content that often constitutes a privacy policy statement. A violation is reported upon completion of the web application crawl without a successful match against any of the web pages.
Note that the privacy policy of your application could be located on another host or within a section of the site that was not configured as part of the scan. To validate, please try to access the privacy policy of your website and check to see if it was part of the scan.

Implication:

Most privacy laws are created to protect residents who are users of the website. Hence, organizations from any part of the world must adhere to these laws if they cater to customers residing in these geographical areas. Failing to do so could result in a lawsuit by the corresponding government against the organization.

Fix:

Declare a comprehensive privacy policy for the website, and ensure that it is accessible from every page that seeks personal information from users. To verify the fix, rescan the site in order to discover and audit the newly added resources.

Descriptions:
Any standard web application privacy policy should include the following components:

- A description of the intended purpose for collecting the data.
- A description of the use of the data.
- Methods for limiting the use and disclosure of the information.
- A list of the types of third parties to whom the information might be disclosed.
- Contact information for inquires and complaints.

Reference:

California Online Privacy Protection Act
http://oag.ca.gov/privacy/COPPA

National Conference of State Legislation
http://www.ncsl.org/issues-research/telecom/state-laws-related-to-internet-privacy.aspx

Gramm-Leach-Bliley Act
http://www.gpo.gov/fdsys/pkg/PLAW-106publ102/pdf/PLAW-106publ102.pdf

Health Insurance Portability and Accountability Act of 1996
https://www.cms.gov/Regulations-and-Guidance/HIPAA-Administrative-Simplification/HIPAAGenInfo/downloads/HIPAALaw.pdf

Health Insurance Portability and Accountability Act of 1996
http://ec.europa.eu/justice/policies/privacy/docs/guide/guide-ukingdom_en.pdf

File Names:
- https://portalproyectos.reniec.gob.pe:443/PLPI

Best Practice **Privacy Violation: Autocomplete**

Summary:

Most recent browsers have features that will save form field content entered by users and then automatically complete form entry the next time the fields are encountered. This feature is enabled by default and could leak sensitive information since it is stored on the hard drive of the user. The risk of this issue is greatly increased if users are accessing the application from a shared environment. Recommendations include setting autocomplete to "off" on all your forms.

Reference:

Microsoft:
Autocomplete Security

File Names:

- https://portalproyectos.reniec.gob.pe:443/PLPI/sendLogin
- https://portalproyectos.reniec.gob.pe:443/PLPI/login?lg=1
- https://portalproyectos.reniec.gob.pe:443/PLPI/inicio
- https://portalproyectos.reniec.gob.pe:443/PLPI

Best Practice

Web Server Misconfiguration: Insecure Content-Type Setting

Summary:

The Content-Type HTTP response header or the HTML meta tag provides a mechanism for the server to specify an appropriate character encoding for the response content to be rendered in the web browser. Proper specification of the character encoding through the charset parameter in the Content-Type field reduces the likelihood of misinterpretation of the characters in the response content and ensure reliable rendering of the web page.Failure to ensure enforcement of the desired character encoding could result in client-side attacks like Cross-Site Scripting.

Execution:

Verify the character set specification on every HTTP response. Character sets can be specified in the HTTP header or in an HTML meta tag. In the case of an XML response, the character set can be specified along with the XML Declaration.

Implication:

In the absence of the character set specification, a user-agent might default to a non-standard character set, or could derive an incorrect character set based on certain characters in the response content. In some cases, both these approaches can cause the response to be incorrectly rendered. This may enable other attacks such as Cross-site Scripting.

Fix:

Ensure that a suitable character set is specified for every response generated by the web application. This can be done either by,

- Modifying the code of the web application, which would require all pages to be modified.
- Adding Content-Type header to the server configuration (**recommended**). This ensures that the header is added to all the responses with minimal development effort.

Reference:

DoD Application Security and Development STIG
http://iase.disa.mil/stigs/app_security/app_sec/app_sec.html

UTF-7 encoding used to create XSS attack
http://www.securityfocus.com/archive/1/420001

File Names:

- https://portalproyectos.reniec.gob.pe:443/PLPI/consultaPadron/cargar/